Pendências técnicas implementadas

Spirit Gate | 18/10/2026 | Java 17

Escopo e origem

Após solicitação para implementar os planejamentos registrados, foram revisados a conversa, README, histórico e relatórios TASK-002/TASK-003. Não foi encontrado arquivo separado de memória acessível. A confirmação do usuário foi implementar as pendências, e não somente organizar o planejamento.

Quatro frentes concretas foram identificadas: validação numérica de movimento; fila de comandos para pressionamentos rápidos; colisão contínua reutilizável para futuros deslocamentos rápidos; busca espacial para reduzir consultas entre entidades. A TASK-003 já estava implementada e foi preservada. Multiplayer, skills novas e alterações de balanceamento não foram introduzidos.

A-01 - Validação numérica

MovementValidation concentra validação de números finitos, valores não negativos e dimensões positivas. CollisionService.isFree agora lança IllegalArgumentException para coordenadas NaN ou infinitas e raio inválido. resolve e resolveContinuous validam origem, ambos os deslocamentos e os resultados de ambos os eixos antes da primeira mutação. Assim um Y inválido não pode deixar X parcialmente aplicado.

MovementService valida direção, velocidade e delta. Overflow nas multiplicações é recusado pela validação do deslocamento resultante. A normalização preserva arredondamento anterior para magnitudes normais e usa double em extremos e subnormais, evitando vetores zero ou não unitários por overflow/underflow. Obstáculos, corpos e dimensões do cenário também são validados.

Os cálculos de interseção mantêm a aritmética anterior quando representável e usam double como fallback em magnitudes extremas. Position continua sendo o valor genérico do domínio, sem modificação. A validação fica nas fronteiras dos serviços de movimento.

Fila de comandos e toques rápidos

SimulationCommandQueue é uma FIFO genérica, independente de Swing e de rede. submit e drain são sincronizados: a simulação recebe um lote imutável e entradas posteriores ficam para o próximo tick. Nenhum produtor altera entidades.

DebugCommandBuffer conserva transições completas de DebugInput e comandos de reinício na ordem em que foram publicados. O runtime usa esse lote em vez de sobrescrever a entrada em AtomicReference. O mundo continua sendo alterado somente pela thread de simulação; o EDT apenas publica comandos.

Cada transição solto/pressionado de E é contada, inclusive múltiplas pressões entre dois ticks. Pressão e soltura rápidas não se perdem. Repetição automática de evento com E mantido não cria outra ativação. Cada ativação passa pelo custo e pela elegibilidade

existentes; recusa não consome e exige nova pressão. A regeneração continua ocorrendo uma vez por tick, nunca por comando.

Movimento e ataque mantidos continuam ativos. Toques completos de movimento produzem intenção por um tick; intenções opostas se cancelam como WASD simultâneo. Ataques rápidos chegam à elegibilidade do ataque e continuam respeitando cooldown. A soltura normal de uma tecla mantida não adiciona outro tick artificial de movimento. O construtor antigo de DebugInput e DebugWorld.tick continuam compatíveis.

Reinícios são comandos ordenados: apagam ativações anteriores ao reset; E mantido exige nova transição. Pressões novas depois do reset permanecem no lote. Testes exercitam a ligação real de E no EDT, inclusive toque completo antes de iniciar a simulação.

A-02 - Varredura contínua disponível

CollisionService.resolveContinuous e MovementService.moveContinuous fornecem a operação para futuros dash e velocidades altas. O algoritmo calcula analiticamente o primeiro contato por eixo: X antes de Y, incluindo círculos contra retângulos com cantos arredondados, círculos contra círculos, tangência e limites do mapa. Mortos e o próprio ator não bloqueiam. A resolução para no contato, sem testar apenas o destino nem depender de amostragem por subpassos.

A origem deve estar livre; penetração inicial lança IllegalStateException sem deslocar. Arredondamentos de contato para dentro são recuados em ULPs; se a posição calculada continuar inválida, o eixo conserva a origem. Os cálculos representam um trajeto X seguido de Y, não uma varredura vetorial simultânea dos dois eixos.

A arena mantém resolve discreto para conservar a mecânica aprovada. A operação contínua já está implementada e testada, mas não cria dash nem modifica o alcance de ataque. Chamadores com movimentos rápidos devem usar explicitamente moveContinuous. O tunelamento do caminho discreto continua sendo uma característica de compatibilidade, não uma garantia do novo caminho contínuo.

A-04 - Busca espacial e contexto persistente

SpatialBodyIndex usa uma grade uniforme de centros. A consulta expande a área pelo maior raio, produzindo uma seleção conservadora; os testes exatos continuam em CollisionService. Não há alteração das regras de colisão. Atualizações após resolve ou resolveContinuous mudam somente a célula do corpo que se moveu. Corpos mortos podem permanecer no índice e são descartados na elegibilidade.

DebugWorld monta Space e a composição dos corpos uma vez por mundo, com células de 96 unidades. A simulação reutiliza esse contexto para jogador e mobs; cada movimento atualiza o índice antes do próximo ator. Isso elimina a reconstrução de listas a cada movimento. Consultas que abrangeriam mais de 4.096 células usam fallback para todos os corpos, sem percorrer bilhões de células vazias.

O construtor anterior de Space com quatro argumentos permanece disponível com busca linear, consultando posições ao vivo. O contexto indexado exige que mutações externas de

posição, como teleporte, chamem index.update(entity). A composição de corpos e seus raios é fixa: adicionar/remover entidades exige reconstruir o contexto nesta versão. O índice cobre corpos; a lista pequena de obstáculos permanece linear.

A-03 - Replay e preservação

MovementReplayTest e seus três baselines não foram alterados. Continuam comparando os campos antigos do snapshot em 1.800 ticks. CombatService, vigor, animação de corte, direção, cone, dano e cooldown mantiveram suas regressões aprovadas. Mudanças futuras na representação dos campos antigos ainda exigem justificativa para atualizar o replay; nenhum hash foi trocado para obter sucesso.

Validação executada

Estado inicial: mvn clean test -q, 30 testes aprovados.

Estado final: mvn clean test, BUILD SUCCESS, compilação release 17, 53 testes, zero falhas, zero erros e zero ignorados.

- MovementHardeningTest: 11 testes, com NaN/infinito, validação atômica, overflow, negativos, geometria inválida, magnitudes extremas, paredes finas, movimento nos dois sentidos, cantos, tangência, entidades, limites e origem penetrada.
- SpatialBodyIndexTest: 5 testes. Mil consultas com 400 corpos coincidem com busca linear. A consulta local retorna menos de 20 candidatos em vez de 400. Inclui atualização, teleporte, morte, varredura indexada, raio grande e fallback.
- DebugCommandBufferTest: 6 testes, com toque rápido, múltiplas pressões, repetição, soltura normal, reinício ordenado, FIFO e dois produtores concorrentes sem perda de comandos.
- StaminaInterfaceTest: 3 testes, incluindo um toque real de E no EDT antes do primeiro tick.
- Outros 28 testes anteriores preservados: movimento, domínio do vigor, combate, replay e integração do mundo.

O teste contínuo amostra os segmentos X/Y de 100 trajetos aleatórios determinísticos para verificar ausência de penetração. Os demais testes verificam contatos analíticos diretamente. A redução de candidatos é verificada funcionalmente; não se afirma uma taxa de frames para centenas de jogadores sem um benchmark de servidor.

Arquivos modificados e criados

Sob server/src/main/java/com/projetok/:
- domain/movement/MovementValidation.java (novo).
- domain/movement/SpatialBodyIndex.java (novo).
- domain/movement/CollisionService.java.
- domain/movement/MovementService.java.
- domain/command/SimulationCommandQueue.java (novo).
- prototype/DebugCommandBuffer.java (novo).
- prototype/DebugRuntime.java.
- prototype/DebugWorld.java.

Sob server/src/test/java/com/projetok/:
- domain/movement/MovementHardeningTest.java (novo).
- domain/movement/SpatialBodyIndexTest.java (novo).
- prototype/DebugCommandBufferTest.java (novo).
- prototype/StaminaInterfaceTest.java.

Documentação: README-PROTOTIPO.md, docs/HISTORICO-TECNICO.md, este relatório, PENDENCIAS-testes.txt, PENDENCIAS.diff e Spirit_Gate_Pendencias_Tecnicas.pdf.
Ferramenta: tools/gerar_pdf_pendencias.py.

Limites da entrega e integração

A fila é para entrada local e não implementa protocolo, autenticação ou política de admissão de comandos de clientes. Ela não cria uma segunda thread de simulação. Não houve alteração de PostgreSQL, Kryonet, pom.xml, CombatService ou regras de stamina.

Os documentos antigos permanecem como registros históricos; limitações ali descritas devem ser lidas junto a esta atualização. Não houve commit, merge ou teste em branch de destino, pois esta pasta não é um checkout Git acessível. A conferência interativa no Windows permanece necessária para a integração; testes sem JFrame não a substituem. Não foi realizada uma sessão interativa nesta etapa.

O diff textual é relativo ao estado inicial desta solicitação. Ele não inclui server/target, binários ou o próprio diff recursivamente. O PDF é entregue separadamente.

Reproduzir: mvn clean test na pasta server. Gerar PDF: python -B tools/gerar_pdf_pendencias.py na pasta Projeto-K.